



INTRODUCTION TO SOFTWARE ENGINEERING

Part II – Software Process Implementation

Software Engineering Department
College of Information and Communication Technology

Content

- Requirements analysis & specification (Definition)
- Design
- Implementation (Programming)
- Testing
- Delivery and Maintenance

Content

- Programming Standards and Procedures
- Programming Guidelines
- Documentation

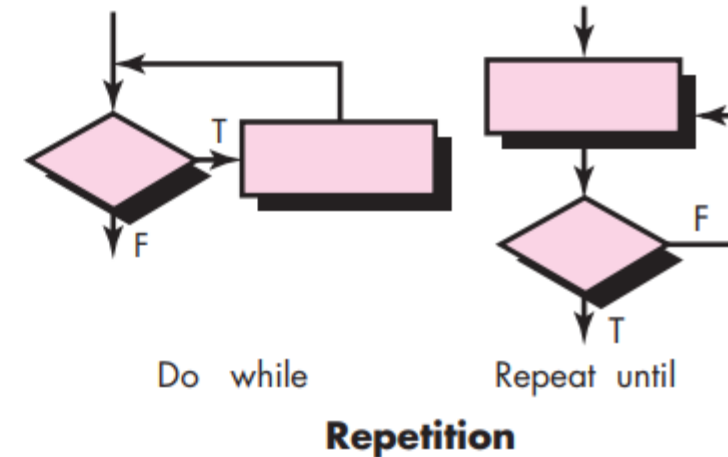
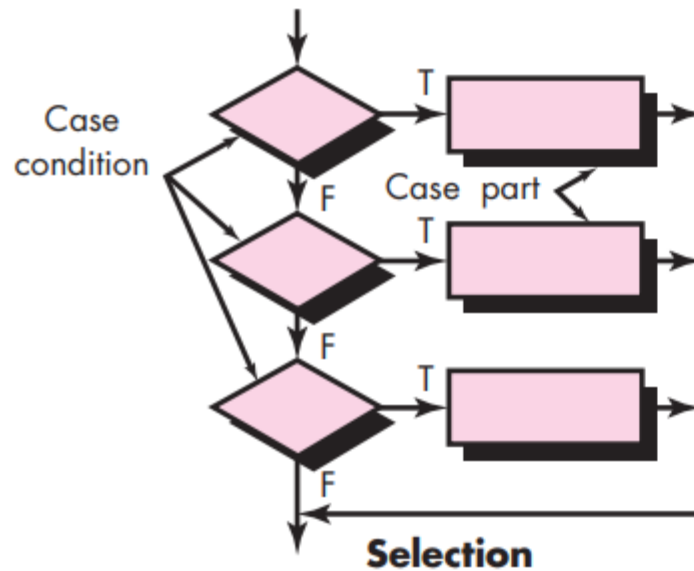
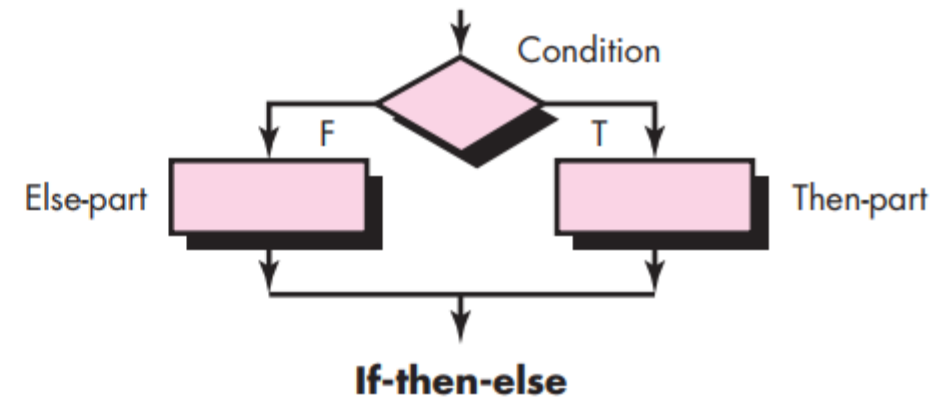
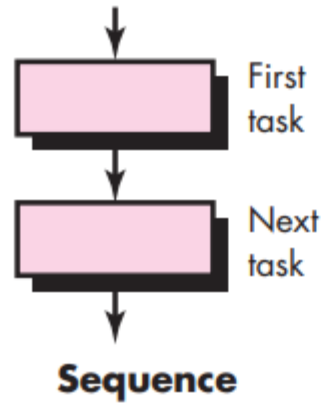
Programming Standards and Procedures

- Standards for you
 - Methods of code documentation
- Standards for others
 - Integrators, maintainers, testers
 - Prologue documentation
 - Automated tools to identify dependencies
- Matching design with implementation
 - Low coupling, high cohesion, well-defined interfaces

Programming Guidelines – Control Structures

- Make the code easy to read
- Build the program from modular blocks
- Make the code not too specific, and not too general
- Use parameter names and comments to exhibit coupling among components
- Make the dependency among components visible

Programming Guidelines – Control Structures



- Control skips around among the program's statements

benefit = minimum;

if (age < 75) goto A;

benefit = maximum;

goto C;

if (age < 65) goto B;

if (age < 55) goto C;

A: if (age < 65) goto B;

benefit = benefit * 1.5 + bonus;

goto C;

B: if (age < 55) goto C;

benefit = benefit * 1.5;

C: next statement

Example of Control Structures

- Rearrange the code

if (age < 55) benefit = minimum;

else if (age < 65) benefit = minimum * 1.5;

else if (age < 75) benefit = minimum * 1.5 + bonus;

else benefit = maximum;

Programming Guidelines – Algorithms

- Common objective and concern: performance (speed)
- Efficiency may have hidden costs
 - cost to write the code faster
 - cost to test the code
 - cost to understand the code
 - cost to modify the code

Programming Guidelines – Algorithms

Name	Best	Average	Worst	Memory	Stable	Method	Other notes
Quicksort	$n \log n$	$n \log n$	n^2	$\log n$	No	Partitioning	Quicksort is usually done in-place with $O(\log n)$ stack space. ^{[5][6]}
Merge sort	$n \log n$	$n \log n$	$n \log n$	n	Yes	Merging	Highly parallelizable (up to $O(\log n)$ using the Three Hungarians' Algorithm). ^[7]
In-place merge sort	—	—	$n \log^2 n$	1	Yes	Merging	Can be implemented as a stable sort based on stable in-place merging. ^[8]
Introsort	$n \log n$	$n \log n$	$n \log n$	$\log n$	No	Partitioning & Selection	Used in several STL implementations.
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	No	Selection	
Insertion sort	n	n^2	n^2	1	Yes	Insertion	$O(n + d)$, in the worst case over sequences that have d inversions.
Block sort	n	$n \log n$	$n \log n$	1	Yes	Insertion & Merging	Combine a block-based $O(n)$ in-place merge algorithm ^[9] with a bottom-up merge sort.
Selection sort	n^2	n^2	n^2	1	No	Selection	Stable with $O(n)$ extra space, when using linked lists, or when made as a variant of Insertion Sort instead of swapping the two items. ^[10]
Bubble sort	n	n^2	n^2	1	Yes	Exchanging	Tiny code size.

https://en.wikipedia.org/wiki/Sorting_algorithm

Programming Guidelines – Data Structures

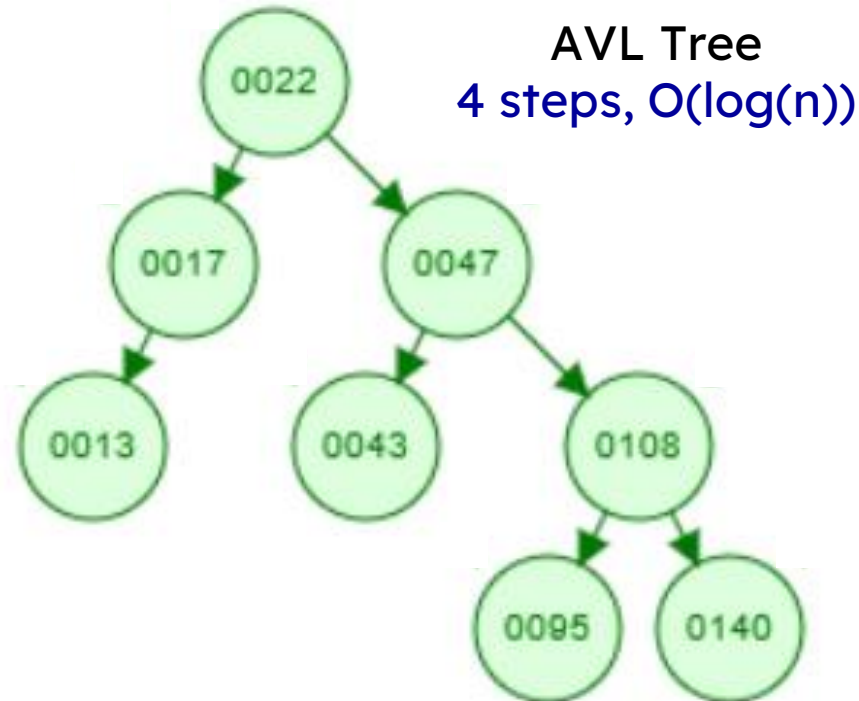
- Several techniques that used the structure of data to organize the program
 - keeping the program simple
 - using a data structure to determine a program structure

Programming Guidelines – Data Structures

- Find 140 in series 22, 43, 17, 95, 13, 47, 108 and 140?

Array
8 steps, $O(n)$

22	43	17	95	13	47	108	140
----	----	----	----	----	----	-----	-----



0	22
1	
2	13
3	47
4	
5	
6	17
7	95
8	140
9	108
10	43

Hash Table
hash function
 $h(x)=x\%11$
1 step, $O(1)$

1. For the first \$10,000 of income, the tax is 10%
2. For the next \$10,000 of income above \$10,000, the tax is 12 percent
3. For the next \$10,000 of income above \$20,000, the tax is 15 percent
4. For the next \$10,000 of income above \$30,000, the tax is 18 percent
5. For any income above \$40,000, the tax is 20 percent

```
tax = 0.  
if (taxable_income == 0) goto EXIT;  
if (taxable_income > 10000) tax = tax + 1000;  
else{  
    tax = tax + .10*taxable_income;  
    goto EXIT;  
}  
if (taxable_income > 20000) tax = tax + 1200;  
else{  
    tax = tax + .12*(taxable_income-10000);  
    goto EXIT;  
}
```

```
if (taxable_income > 30000) tax = tax + 1500;  
else{  
    tax = tax + .15*(taxable_income-20000);  
    goto EXIT;  
}  
if (taxable_income < 40000){  
    tax = tax + .18*(taxable_income-30000);  
    goto EXIT;  
}  
else  
    tax = tax + 1800. + .20*(taxable_income-40000);  
EXIT;
```

**Programming Guidelines-
Keep the Program Simple**

Programming Guidelines

- Define a tax table for each “bracket” of tax liability

Bracket	Base	Percent
0	0	10
10,000	1000	12
20,000	2200	15
30,000	3700	18
40,000	55000	20

- Simplified algorithm

```
for (int i=2; level=1; i <= 5; i++)
```

```
    if (taxable_income > bracket[i])
```

```
        level = level + 1;
```

```
tax= base[level]+percent[level] * (taxable_income - bracket[level]);
```

Programming Guidelines – General Guidelines to Preserve Quality

- Localize input and output
- Employ pseudocode
- Revise and rewrite, rather than patch
- Reuse
 - Producer reuse: create components designed to be reused in future applications
 - Consumer reuse: reuse components initially developed for other projects

Programming Guidelines

Consumer Reuse

- Four key characteristics to check about components to reuse
 - does the component perform the function or provide the data needed?
 - is it less modification than building the component from scratch?
 - is the component well-documented?
 - is there a complete record of the component's test and revision history?

Programming Guidelines

Producer Reuse

- Several issues to keep in mind
 - make the components general
 - separate dependencies (to isolate sections likely to change)
 - keep the component interface general and well-defined
 - include information about any faults found and fixed
 - use clear naming conventions
 - document data structures and algorithms
 - keep the communication and error-handling sections separate and easy to modify

Documentation

- Internal documentation
 - header comment block
 - meaningful variable names and statement labels
 - other program comments
 - format to enhance understanding
 - document data (data dictionary)
- External documentation
 - describe the problem
 - describe the algorithm
 - describe the data

Documentation

Information Included in Header Comment Block

- What is the component called
- Who wrote the component
- Where the component fits in the general system design
- When the component was written and revised
- Why the component exists
- How the component uses its data structures, algorithms, and control

Q&A

References

- Shari Lawrence Pfleeger, Joanne M. Atlee, Software Engineering theory and practice, Pearson Prentice Hall, 4th edition, 2010.
- Hiep Xuan Huynh, Lan Phuong Phan, Introduction to Software Engineering, Can Tho university publishing house, 2010.